GENERALIZATION MECHANICS IN DEEP LEARNING

Anonymous authors
Paper under double-blind review

ABSTRACT

Deep neural nets are well known for their ability to generalize, but understanding the mechanics of generalization remains an important open question. We analyze the probability-flow through a deep net, especially what happens to this flow during the training process. Quantitatively, this study leads to a new approach for obtaining generalization bounds, using which we derive a non-vacuous bound for an over-parameterized deep network that learns relatively small dataset such as CIFAR10. On the qualitative side, this reveals that the process of learning imposes an implicit regularization, namely orbit-regularization, as an essential ingredient for generalization.

1 INTRODUCTION

Despite their far-reaching success, why deep nets work so well still remains a mystery and a quest. During its initial phase this quest was scattered among different lines of work - such as understanding regularization effects (implicit or explicit), fitting deep learning to mathematical models understood elsewhere (*e.g.* statistical physics) and so on. After Zhang et al. (2017) demonstrated that deep nets tend not to overfit even when they are capable of doing so - for they can easily fit high degree of noise - search for generalization bounds took the limelight. Since then, a long line of generalization bounds have appeared, both vacuous and non-vacuous, chipping away at this puzzle from different angles. Nevertheless, a complete picture is yet to emerge.

Generalization is typically explored in quantitative terms - *viz.,* how much true risk can deviate from what is empirically observed. However, there is a qualitative aspect that's often lost. In the vast space of configuration parameters, what is special about the one that emerges as the output of SGD? Unfortunately a chasm persists between the world of quantitative estimation and that of qualitative and explanatory characterization. The means to bridge this gap would be to understand what structural features SGD extracts that naturally connect to generalization (Dziugaite, 2020) - which, to date has remained elusive.

In this paper, we propose a new approach - *viz.,* analyzing the probability-flow through a deep net - that maybe helpful in bridging the gap. On one hand, we show that examining these probability-masses empirically helps us derive non-vacuous bounds. On the other hand, and perhaps this is more important, this study leads to a specific characterization of configuration parameters that emerge as SGD's outcome; a characterization that enjoys a more natural semantic alignment with the notion of generalization than previously studied quantities (such as norm, margin, flatness of minima *etc*).

RELATED WORK

There is a long list of recent theoretical studies, each trying to expose the mechanics of deep learning and architectures from different angles (a small sampler: (Vidal et al., 2017; Arora et al., 2015; Gilbert et al., 2017; Patel et al., 2016; Schoenholz et al., 2016; Poole et al., 2016; Tishby & Zaslavsky, 2015; Arora et al., 2018a; Gunasekar et al., 2018; Hardt & Ma, 2016; Soudry et al., 2018)).

In the post-Zhang era, the thrust shifted towards generalization, and especially on deriving upper bound on generalization. This corpus of work is growing very rapidly at the moment (a tiny sampler: (Neyshabur et al., 2018; Poggio et al., 2018; Zhou et al., 2018; Long & Sedghi, 2019; Arora et al., 2018b; Dziugaite & Roy, 2017; Bartlett & Long, 2021; Liao et al., 2020; Nagarajan & Kolter, 2019b)). However, these generalization bounds have their own limitations; for instance some are vacuous to begin with; others, while non-vacuous in some settings, may turn vacuous under many

different conditions. Some of these bounds can only be applied to variants of the original networks. Yet another class of them, very surprisingly, grow with increasing size of training sets. Nagarajan (2021) presents a well curated survey of these bounds, especially where and how they stumble, and it is conjectured that uniform convergence, a common technique shared by these bounds, might be the root cause behind their failure modes. See (Bartlett et al., 2021) for a more in-depth statistics perspective on generalization and related phenomena such as overfitting.

CONTRIBUTIONS OF THIS PAPER

Perhaps the main contribution of this paper is to highlight the possibility that analyzing the probability distributions that flow through a neural net may be profitable for various purposes. Over sections 2.2 and 2.4 we establish that by breaking down the seemingly complex structure of a neural net into its simpler building blocks, and then examining how these blocks are essentially quite limited in their ability to create significant deviation between the empirical and true distributions that flow through the net, it becomes possible to obtain generalization bounds. In section 2.5 we provide experimental validation of this non-vacuous bound. In the subsequent sections 3 and 4 we show - again, by inspecting how the probability masses are morphed - that the mechanics of backpropagation specifically promotes filters that have higher invariance to the intrinsic variations of the training dataset. This characterization indeed has a direct semantic connection to generalization, and falls right out of the probability flow analysis.

2 GENERALIZATION BOUNDS FROM THE PROBABILITY FLOW

2.1 NOTATIONS AND KNOWN OBSTRUCTIONS

First we set up the required notation and then briefly recap the difficulty of deriving a generalization bound for deep nets at a high level. We are considering a standard supervised classification scenario.

Let Ω be the sample space, and $\mathcal{D}$ is a data distribution over Ω. A single element $\omega \in \Omega$ has the form $\omega = (x, y)$, where x is the data element, and y is the label. Let $\mathcal{H}$ be the hypothesis-class. Let $S = (\omega^{(1)}, \omega^{(2)} \dots \omega^{(n)})$ be a sample of n data points drawn *i.i.d* from $\mathcal{D}$, $h_S \in \mathcal{H}$ the outcome of the learning algorithm. For $\omega \in \Omega$ (*i.e.*, $\omega = (x, y)$) and hypothesis $h \in \mathcal{H}$, $\mathcal{L}(h(x), y)$ denotes the loss function. Define $\mathcal{L}_{\mathcal{D}}(h_S) := \mathbb{E}_{\omega \sim \mathcal{D}}[\mathcal{L}(h_S(x), y)]$ (true loss) and $\hat{\mathcal{L}}_S(h_S) := \frac{1}{n} \sum_{\omega^{(i)} \in S} \mathcal{L}(h_S(x^{(i)}, y^{(i)}))$ (empirical loss). The generalization error then, is defined by the smallest number $\epsilon_{gen}(n, \delta)$ such that the following holds:

$$\mathbf{Pr}_{S \sim \mathcal{D}^n}[\mathcal{L}_{\mathcal{D}}(h_S) - \hat{\mathcal{L}}_S(h_S) \leq \epsilon_{gen}] \geq 1 - \delta \tag{1}$$

Since the hypothesis class could be quite complex, and the actual data distribution $\mathcal{D}$ is unknown, it is impossible to get a direct handle on the true loss $\mathcal{L}_{\mathcal{D}}(h_S)$ in the above equation. The historically older technique of taming this quantity involved taking a union-bound over the hypothesis class, along with a standard concentration inequality (such as Hoeffding's) yielding a relationship of the form: $\mathbb{E}_{S \sim \mathcal{D}^n} |\mathcal{L}(h) - \hat{\mathcal{L}}(h)| \leq C.\mathcal{G}(n).\exp(-2n\epsilon^2)$, where $\mathcal{G}(n)$, the growth-function of the hypothesis class, typically dominates the decay term in over-parameterized regimes, immediately rendering the bound vacuous.

To circumvent the obstruction presented by monstrously big growth functions, more recent techniques adopted, in some way or other, a line of argument through *uniform convergence*, that (roughly) attempts to prove the following statement :

$$\mathbf{Pr}\left[\sup_{S \sim \mathcal{D}^n} \sup_{h \in \mathcal{H}} \left| \mathcal{L}_{\mathcal{D}}(h) - \hat{\mathcal{L}}_S(h) \right| \leq \epsilon_{unif} \right] \geq 1 - \delta \tag{2}$$

The smallest such constant $\epsilon_{unif}(n, \delta)$ meeting the above criterion becomes a bound on the generalization error. However, this line of work also landed into various troubles, and might be inherently limited. This issue was laid out in a recent work by Nagarajan and Kolter Nagarajan & Kolter (2019a); in particular they showed that it may be always possible to find one bad test-set with a large error, regardless of the training set and the learned hypothesis. Even if this test-set surfaces with a

low probability, that does not matter because its large error is picked up while taking the supremum in equation 2, yielding $\epsilon_{unif} \approx 1$, and therefore a (near) vacuous bound.

2.2 UNIFORM LAWS AND OVER FITTING

The fact that an extremely sophisticated corpus of techniques for obtaining generalization bounds came into existence over many decades, is for one simple reason: standard concentration inequalities cannot be readily applied to the output of a learning algorithm. This, in turn, happens because uniform laws of large numbers may not hold in general for an arbitrary empirical risk minimizer (Wainwright, 2019). To that end, we establish why it is reasonable to expect that a neural net is not likely to breach uniform laws, except special circumstances.

The primary version of such a uniform law states that the empirical cumulative distribution functions(CDF) of a random variable converges to its actual CDF in probability. This is the classical *Glivenko-Cantelli* theorem (Wainwright, 2019), originally established for univariate situation and in an asymptotic sense ($n \to \infty$). This result was subsequently extended in stages, first for non-asymptotic cases, then over many variables. Here, we use this extended version, known as the *multivariate Dvoretzky-Kieffer-Wolfowitz theorem*, derived by Naaman (2021) :

Theorem 2.1. *Let $X \in \mathbb{R}^d$ be a random variable, with cdf denoted by $\mathbf{F}^X(x) \coloneqq \mathbf{Pr}[X \leq x]$. Let the empirical cdf be $\mathbf{F}_n^X(x) \coloneqq \sum_{j=1}^{n} \mathbb{1}_{(x^{(j)} \leq x)}$. Then the following holds:*

$$\mathbf{Pr}\left[\sup_{t \in \mathbb{R}^d} \left|\mathbf{F}_n^X(t) - \mathbf{F}^X(t)\right| \geq \epsilon\right] \leq \overbrace{d(n+1)\exp(-2n\epsilon^2)}^{\delta} \tag{3}$$

Given δ and n, we get $\epsilon_{\texttt{dkw}}(n, \delta) = \sqrt{\frac{1}{2n}\log\frac{d(n+1)}{\delta}}$ as an upper bound on the uncertainty of estimating a CDF. In order to apply the above notion of uncertainty/reliability across different layers of a neural net, we rephrase it into a parameterized form, as captured by the following definition.

Definition 1 ($\mathbf{GC}(\epsilon, \delta)$-reliability)**.** Let $X \in \mathbb{R}^d$ be a random variable. We say that the distribution of X - or simply, X - is $\mathbf{GC}(\epsilon, \delta)$-reliable if the following holds true for its CDF:

$$\mathbf{Pr}\left[\sup_{t \in \mathbb{R}^d} \left|\mathbf{F}_n^X(t) - \mathbf{F}^X(t)\right| \geq \epsilon\right] \leq \delta \tag{4}$$

First we outline our strategy in an abstract and idealized (albeit somewhat trivial) setup.

Example 1 (An abstract example). Let $\Omega \subseteq \mathbb{R}^d$ and $\mathcal{S} = \{H | H \text{ is a finite subset of } \Omega\}$, $\psi, \chi : \Omega \to \{0, 1\}$. We are given that ψ is the indicator function for a bounded measurable subset $A \subseteq \Omega$, *i.e.*, $\psi(x) = \mathbb{1}_{x \in A}$. $\chi \in \{\mathbb{1}_H | H \in \mathcal{S}\}$ - *i.e.*, χ is chosen from an infinitely large family of functions that can create δ-function peaks over any arbitrary but finite set of points. And let $\Phi(x) = \max\{\psi(x), \chi(x)\}$. Finally let $X \sim \mathcal{D}(\Omega)$ be a random-variable sampled from a density $\mathcal{D}$ over Ω. The goal is to estimate the distribution $\mathbf{F}^Y(y)$ of $Y = \Phi(X)$ through a finite set of samples $S = (x^{(1)}, x^{(2)} \dots x^{(m)}) \sim \mathcal{D}^m$.

In general, as long as we treat Φ as a black box, we cannot estimate the distribution $F^{\Phi(X)}$ via finitely many samples, because, *in theory*, it is always possible that $\chi(x)$ can fit every element of any finite S. However, for any continuous probability density that has no atom ($\mathbf{Pr}[\{x\}] = 0$, for any $x \in \Omega$), it is also true that $\mathbf{Pr}[\chi(x) = 1] \approx 0$. Therefore, if we *somehow* know χ, given any S, we can compute the set $S' = \{x^{(i)} \in S | \chi(x^{(i)}) = 0\}$. Let $m' = |S'|$. Using these m' samples, we can still estimate a bound on $|F^\Phi - F^\Phi_{m'}|$ via DKW-inequality 3. Informally, we are first eliminating the samples that are *possibly overfit* by χ, and then estimating the distributions based on ψ only. Using fewer samples *could* lead to a looser bound, but it also ensures *erring* on the side of caution leading to a more conservative estimate.

Formally speaking, based on the new sample-count m', we have a constant $\epsilon' = \epsilon(m', \delta)$, such that Y is $\mathbf{GC}(\epsilon', \delta)$-reliable. If $m' \approx m$, then our bound, ϵ', is as tight as allowed by DKW-inequality on F^X. On the other hand, if $m' \approx 0$, *i.e.*, most of the samples had to be dropped, then we get a very large value of ϵ' (possibly vacuous, even $\gg 1$), and hence a loose bound. This is both desirable and expected, for if most of the samples are *overfit* via χ, the empirical estimate of Y must not be reliable. We capture this formally in the following definition.

Definition 1 (Overfitting). *Let X be a a random variable with distribution $\mathcal{D}$, and $Y = \chi(X)$. We say that the function χ overfits a sample x_i, if for any set $S \sim \mathcal{D}^m$, as long as $x_i \in S$, it is impossible to establish that Y is $\mathbf{GC}(\epsilon, \delta)$-reliable for any $\epsilon, \delta \leq 1$.*

Note that the above characterization is purely theoretical, in the sense that in general there may not be a way to establish whether an arbitrary function χ *overfits* a training sample or not. However, in the event it is possible to identify samples that are potential overfit - as illustrated in example 1 - we might be able to prove $\mathbf{GC}(\epsilon, \delta)$-reliability after removing these samples from our count. On the other hand, proving that a function is $\mathbf{GC}(\epsilon, \delta)$-reliable ensures, by definition, that no sample is overfit.

2.3 Overfitting at a Single Layer

Let us consider a single layer of neurons, $\phi(X) = \sigma(WX - \mathbf{b})$. $X \in \mathbb{R}^d$ is the input random variable, assumed to be $\mathbf{GC}(\epsilon, \delta)$-reliable. $W = [\theta_1, \theta_2 \dots \theta_{d'}]$ is the weight matrix and σ is the non-linearity, hereon assumed to be ReLU. The column vectors $\theta_k \in \mathbb{R}^d$, and the bias vector $\mathbf{b} = (b_1, b_2 \dots b_{d'}) \in \mathbb{R}^{d'}$. Define $Y \coloneqq WX$, *i.e.*, $Y_k \coloneqq {\theta_k}^T X$, and $Z_k \coloneqq \sigma(Y_k)$. Further, let $S = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)} \dots \mathbf{x}^{(n)})$ be a set of samples of X drawn *i.i.d* from its distribution. We assume that the samples are energy-normalized, *i.e.*, $\|\mathbf{x^{(1)}}\|_2 = \|\mathbf{x^{(2)}}\|_2 = \dots \|\mathbf{x^{(n)}}\|_2 = 1$.

We ask : *what does it mean for ϕ to **overfit** training samples?*
To that end, we observe that ϕ has three essential components - (*i*) the linear unit (*ii*) the ReLU component and finally (*iii*) the composition of ReLU with the linear action. In the coming sections we will show that the linear layer, and ReLU, in themselves remain $\mathbf{GC}(\epsilon, \delta)$-reliable. However, it is their composition that could create potential overfits. Here, we identify how that happens, essentially creating a recipe for weeding out samples that are potentially *overfit*. We illustrate this with a simple example.

Example 2 (Single Layer Overfitting). Consider one sample $\mathbf{x^{(j)}}$, and let θ_k be chosen to be fully aligned with it - *i.e.*, $\langle \theta_k, \mathbf{x^{(j)}} \rangle = 1$, but for all $i \neq j$, we have $\langle \theta_k, \mathbf{x^{(i)}} \rangle < 1$. Setting bias in the range $\max_{\{i | i \neq j\}} \{\langle \theta_k, \mathbf{x^{(i)}} \rangle\} < b_k < 1$, and then applying σ results in only sample $\mathbf{x^{(j)}}$ advancing through neuron-k, while $\sigma(\langle \theta_k, \mathbf{x^{(i)}} \rangle - b_k) = 0$ for all other samples $i \neq j$. In particular, when $d' > n$, it is entirely possible that each sample $\mathbf{x^{(j)}}$ advances through some neuron k - *i.e.*, $\sigma(\langle \theta_k, \mathbf{x^{(j)}} \rangle - b_k) > 0$ - and yet blocked everywhere else, *i.e.*, $\sigma(\langle \theta_l, \mathbf{x^{(j)}} \rangle - b_l) = 0$ for all $l \neq k$. Looking at the output of any single neuron, it would appear that there is little or no chance that any sample advances to the next layer. Yet, with appropriate choices of θ_k and b_k as specified above, the entire set of samples would advance forward, breaching $\mathbf{GC}(\epsilon, \delta)$-reliability. We formalize this informal notion in the following definition.

Definition 2 (κ-overfitting). *For $0 \leq \kappa \leq 1$, a neuron κ-overfits a sample $\mathbf{x^{(j)}}$ if the following conditions hold.* (**1**) *it advances at most κ fraction of the samples and* (**2**) *it advances sample $\mathbf{x^{(j)}}$. A layer (or a whole network) ϕ κ-overfits $\mathbf{x^{(j)}}$ if ϕ has a neuron that κ-overfits $\mathbf{x^{(j)}}$.*

A large κ means a large fraction of samples are advancing - and thus there is no overfitting. For instance, if 10% of the samples advance at a given neuron, then we expect an equal sized fraction (within $\pm\epsilon$) from any other independently drawn sample to advance as well. In fact that remains to be the case for most $\kappa > 0$. It is only for the theoretical situation, $\kappa \to 0^+$, we would get the arbitrarily sharp δ-function peaks over the sample-space, which causes *overfitting* in the sense of definition 1. We remove these overfit samples using the following step.

Definition 3 (κ_0-filtering). *Let $\kappa_0 > 0$ be a small constant. Given a set of training samples S, construct the set $H = \{\mathbf{x^{(i)}} \in S | \exists \kappa \leq \kappa_0$, such that ϕ κ-overfits $\mathbf{x^{(i)}}\}$. Define $S_{\kappa_0} \coloneqq S \setminus H$. We say S_{κ_0} is κ_0-filtered with respect to ϕ.*

Our high-level strategy to establish $\mathbf{GC}(\epsilon, \delta)$-reliability of a single layer ϕ, then, can be summed up in the following proposition.

Proposition 1. *(1) Fix a small constant $\kappa_0 > 0$, and create $S_{\kappa_0} \subseteq S$ that is κ_0-filtered with respect to ϕ. This ensures that there is no overfit sample to tilt the empirical distribution. (2) Establish that the two operations,* viz.*, , linear transform and ReLU, preserve $\mathbf{GC}(\epsilon, \delta)$-reliability if the input*

already has that property. And finally (3) Using $m' = |S_{\kappa_0}|$ determine the appropriate value of the constant ϵ, for any given confidence parameter δ.

2.4 FROM $\mathbf{GC}(\epsilon, \delta)$-RELIABILITY TO GENERALIZATION BOUND

We further flesh out the steps of proposition 1 above. The first step, κ_0-filtering, is already addressed. Next, we will establish $\mathbf{GC}(\epsilon, \delta)$-reliability of a neural network and then derive a generalization bound. However, for this second step, and other requirements in subsequent sections, we need to rewrite the loss function $\mathcal{L}$, essentially decomposing it over individual classes.

Observe that $\mathcal{D}(\omega)$ in equations 1 and 2 is a joint distribution over data and labels ($\omega = (x, c)$), and therefore can be decomposed as $\mathcal{D}(x, c) = \mathcal{D}_c.\mathcal{D}_{x|c}$, where $\mathcal{D}_c = \int_x \mathcal{D}(x, c)$ is the marginal over class-label c and $\mathcal{D}_{x|c} = \mathcal{D}(x, y|y = c)$ is the conditional. Hence, the true loss of a hypothesis h can be refactored as:

$$\mathcal{L}_{\mathcal{D}}(h) = \mathbb{E}_{\omega\sim\mathcal{D}}[\mathcal{L}(h(x), c)] = \mathbb{E}_{c\sim\mathcal{D}_c}\left[\overbrace{\mathbb{E}_{x\sim\mathcal{D}_{x|c}}[\mathcal{L}(h(x), c)]}^{\mathcal{L}^c(h)}\right] = \mathbb{E}_{c\sim\mathcal{D}_c}[\mathcal{L}^c(h)] \tag{5}$$

The refactored expression in equation 5 shows that for any hypothesis h, and for a *fixed* class c, the *functional form* of the true loss $\mathcal{L}^c(h) = \mathbb{E}_{x\sim\mathcal{D}_{x|c}}[\mathcal{L}(h(x), c)]$ is independent of x. This automatically carries over to the empirical loss $\hat{\mathcal{L}}^c(h) = \frac{1}{n}\sum_i \mathcal{L}(h(x^{(i)}), c)$, where each data element $x^{(i)}$ has a fixed class-label c [1]. Therefore, ***if*** the distribution of the variable $z = h(x)$ ***happens*** to be $\mathbf{GC}(\epsilon, \delta)$-reliable, then we can *hope* to upper-bound $|\mathcal{L}^c(h) - \hat{\mathcal{L}}^c(h)|$. Of course, expecting this is a very tall order, because in reality h is learned from the samples, and thus certainly not independent of the inputs. At any rate, from hereon, our analysis proceeds as if we are first dealing with samples from a fixed class c (loss function: $\mathcal{L}^c(h)$). We will take expectation over different classes at the very end.

A hypothesis h (a deep net) is composed of multiple layers: $h(x) = \zeta(\phi_L(..\phi_2(\phi_1(x))))$. We assume a standard structure for every layer : $\phi_l(X_{l-1}) = \sigma[W_l X_{l-1} - b_l]$, where W_l is the weight matrix, b_l is the bias vector and σ is the non-linearity (assumed ReLU). Since, the bias vector can be subsumed inside the weight matrix by introducing another fixed-valued input-dimension (separately for each layer), from hereon, we assume $\phi_l(X_{l-1}) = \sigma[W_l X_{l-1}]$. The final step, ζ, is the *softmax* function applied to the L-th layer ϕ_L. For $Y \in \mathbb{R}^M$, $\zeta_k(Y) = \frac{\exp(Y_k)}{\sum_{j=1}^M \exp(Y_j)}$, for $k \in \{1, 2 \ldots M\}$.

We are ready to derive our generalization bound, in stages, for a neural network h. Based on our discussions in section 2.3, we assume that we already have a distilled set of samples S_{κ_0} that is κ_0-filtered with respect to h. First we establish that linear units preserve $\mathbf{GC}(\epsilon, \delta)$-reliability (lemma 2.2). Next, we claim the same for ReLU units (lemma 2.3). In conjunction with κ_0-filtered samples, these two results can be inductively used to claim that the whole network preserves this property up to layer ϕ_L. The softmax layer weakens this reliability, as lemma 2.4 shows; for a fixed confidence parameter δ, the uncertainty band ϵ expands to $M\epsilon$. Note that this is still a bound on the CDF, before the loss function is evaluated. As the final step, in theorem 2.5 this CDF-bound is used to estimate the bound on generalization error ($\epsilon_{\texttt{gen}}$) which is a function of $\mathcal{L}^c(h)$. All these results are proved in Appendix A. We note here that theorem 2.5 makes mild assumptions on $\mathcal{L}^c$, but they are not very restrictive and satisfied by common functions such as cross-entropy, or what we used for our experiments in section 2.5.

Lemma 2.2. *If X_{l-1} is $\mathbf{GC}(\epsilon, \delta)$-reliable, then $Y = W_l X_{l-1}$ is also $\mathbf{GC}(\epsilon, \delta)$-reliable.*

Lemma 2.3. *If Y is $\mathbf{GC}(\epsilon, \delta)$-reliable, then so is $\sigma(Y)$.*

Lemma 2.4. *Let $Y \in \mathbb{R}^M$ be the last-layer's output. If Y is $\mathbf{GC}(\epsilon, \delta)$-reliable, then $\zeta(Y)$ is $\mathbf{GC}(M\epsilon, \delta)$-reliable.*

[1] This will not be the case for the loss function in its original form. Because in that form, the expectation is taken over ω, and although, the functional form of $\mathcal{L}(h(x), c)$ does not depend on x, it certainly depends on c, which is part of ω. When c is fixed, that part essentially becomes a constant. Of course, now the sample-set changes to examples with class label c only.

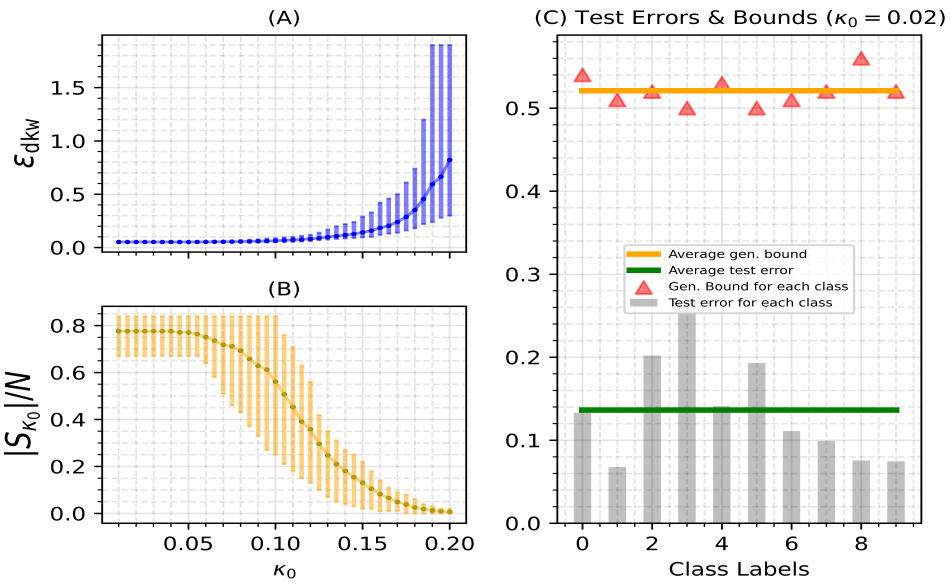


Figure 1: Experiments on a model that learned CIFAR-10 dataset. Plots (A) and (B) share the same x-axis. (A) The average of $\epsilon_{\texttt{dkw}}$ across 10 classes (with error-bars) is plotted against κ_0. (B) The average size (as a fraction) of the corresponding S_{κ_0} is plotted against κ_0. (C) Test errors for different classes shown separately (gray histograms) along with their average (≈ 0.13). Numerically calculated upper bound for each class is presented (red triangles) along with the mean value of $\epsilon_{\texttt{gen}} \approx 0.52$ - which is non-vacuous. $\delta = 0.05$, hence this bound holds with probability $\geq 95\%$.

Theorem 2.5. *Let $Y = h(X)$ be a neural net that learns a M-way classification task. Given a small positive constant κ_0, let S_{κ_0} be a κ_0-filtered set of training samples drawn* i.i.d *from some data-distribution $\mathcal{D}_{x|c}$ (as in equation 5, so that they all have fixed class label c)* , i.e., $S_{\kappa_0} \sim \mathcal{D}^n_{x|c}(\mathbb{R}^d)$. *Let $\ell^c(y)$ be the loss function evaluated on the coordinate Y_c, that satisfies the following properties:* (i) *$\ell^c(y)$ is only finitely supported and* (ii) *over this finite support, $\frac{d}{dy}[\ell^c(y)] < 0$. Then, with probability at least $1 - \delta$ the generalization error $\epsilon_{\texttt{gen}}$ is upper-bounded as follows:*

$$\epsilon_{\texttt{gen}} \leq M \left[|\max\{\ell^c\} - \min\{\ell^c\}| \right] \sqrt{\frac{1}{2n} \log \frac{n(d+1)}{\delta}} \tag{6}$$

2.5 Experiments

We train a wide resnet model on CIFAR-10 dataset. We used a pytorch implementation as supplied by Zhang et al. (2020). This model is over-parameterized (with respect to the dataset), and was originally used by Zhang et al. (2017) to demonstrate that it can fit high degree of noise (label corruption). Once the model is fully trained and attains near 100% accuracy, we obtain a numerical value for bounding $\epsilon_{\texttt{gen}}$. In order to do so, we look at how samples from a single class moves as a distribution (empirically, a point-cloud) through various layers. For any fixed constant κ_0, it is then possible to determine a set of samples S_{κ_0} by peeking into output of every layer and checking if any sample is κ_0-overfitting at any neuron. These overfit samples are then dropped and the remaining set S_{κ_0} is thus κ_0-filtered with respect to the model. Using this set, and using DKW-inequality from theorem 2.1, we can compute a bound $\epsilon_{\texttt{dkw}}$ on the CDFs of the empirical point-clouds (before softmax). And finally, we plug in $\epsilon_{\texttt{dkw}}$ in theorem 2.5 to upper-bound $\epsilon_{\texttt{gen}}$.

We repeat the above experiment for a range over κ_0. When κ_0 is small, and if the network is *not overfitting* (in the sense of definition 1) we expect S_{κ_0} to remain unchanged. As a result, $\epsilon_{\texttt{dkw}}$ remains stable. This is plotted in plot (A) of figure 1. However, trained neurons are supposed to

detect features, and not every sample has every feature learned. Thus for larger values of κ_0, we expect to see the size of S_{κ_0} dwindle quickly, as shown in plot (B) of figure 1. Finally, choosing a κ_0 from a region where $\epsilon_{\texttt{dkw}}$ (and S_{κ_0}) is stable, we can compute numerical bounds on $\epsilon_{\texttt{gen}}$, separately for every class. This is displayed in plot (C) of figure 1.

For these experiments, we used a simplified version of a multi-class loss function, re-written for a fixed class c. The true risk is : $\mathcal{L}^c(h) = \mathbb{E}_{x\sim\mathcal{D}_{x|c}}[1 - \langle h(x), \mathbf{1_c}\rangle]$ and the empirical version : $\hat{\mathcal{L}}^c(h) = \frac{1}{m}\sum_{i=1}^{m}[1 - \langle h(x_i), \mathbf{1_c}\rangle]$. $\mathbf{1_c}$ is the vector that is 1 on class c and zero everywhere. Although the formulation in theorem 2.5 is compatible with more popular loss functions such as cross-entropy, we chose this function because it directly approximates the fraction of samples that are misclassified, and thus we can trivially check if the resulting bound is numerically vacuous.

3 Learning and Contraction of Probability Masses

So far we established that if a set of training samples run through a learned neural network, the empirical point-clouds generated at different layers reliably represent the actual distribution. In this section, we ask: *how learning dynamics changes these distributions?* Surprisingly, the answer reveals how invariance is extracted from training data and encoded into network parameters.

As before, consider a classifier neural network, with L layers, and M output classes. $\mu_l^c(t)$ denotes the distribution of class c at layer l at iteration step t. The sequence: $\mu_0^c(t) \to \mu_1^c(t) \to \ldots \mu_L^c(t)$ then describes a snapshot of how the probability-mass for class c flows from input to the output. The input distribution μ_0^c remains unchanged. The target distribution at the output is generally a point-mass, $\nu^{\mathbf{c}}$, over a point in $\mathbb{R}^M$. Since the target is a point-mass, and gradient-descent by definition attempts to reduce the discrepancy between $\nu^{\mathbf{c}}$ and $\mu_L^c(t)$, it is expected that over time $\mu_L^c(t)$ will be squeezed towards becoming a point mass. However, it turns out that this *squeeze-effect* actually propagates inward and tries to reduce the dispersion of point-clouds at layers $0 < l < L$.

For $X \sim \mathcal{D}(\mathbb{R}^d)$, denote the dispersion of X as $Var(X) = \mathbb{T}\mathbb{r}(\mathbb{E}[(X - \mathbb{E}X)(X - \mathbb{E}X)^T])$. As before, for the remaining discussion, we limit ourselves to a fixed class c. Let X_l denote the random variable representing samples that are *advancing* at layer l (*i.e.*, there exists at least one coordinate j such that $X_{l,j} > 0$). We have the following result on contraction passing inward (proof in Appendix A2)

Theorem 3.1. *During gradient-descent, if the output probability mass contracts at time-step t,* i.e.*, $Var(X_L(t+1)) < Var(X_L(t))$, then this contraction proceeds to inner layers. In other words, even for $1 < l < L$ we get $Var(X_l(t+1)) < Var(X_l(t))$.*

We have also conducted experiments at Imagenet-scale that corroborate the prediction of theorem 3.1. The details of those experiments are described in Appendix A3.

4 Implicit Regularization of Pseudo-Orbits and Feature Selection

Since learning causes contraction of probability-masses, a natural question follows: *can we characterize how contraction affects feature-selection?* We now show why contraction actually promotes features that are in some sense simpler and, perhaps more importantly, have higher invariance with respect to the input-variations.

For a multidimensional probability-mass to contract, there must be contraction over individual dimensions. For each point in this high-dimensional space, every coordinate is generated in the form $\sigma(\langle x, f\rangle)$. Therefore, during backpropagation f would be so adjusted that the distribution of $\langle x, f\rangle$ contracts.

It turns out that a *simpler* filter is more likely to generate a tighter distribution. We motivate our notion of simplicity with a toy experiment. This is described in Figure 2, where a set of random deformations of an image is passed through two filters. In an informal sense the elliptical filter has a simpler shape, and it produces an evidently tighter distribution of the inner-products over the input's variations. Thus, an optimization scheme trying to contract the inner-product distribution would choose this filter over the other that, one could argue, looks more complex.

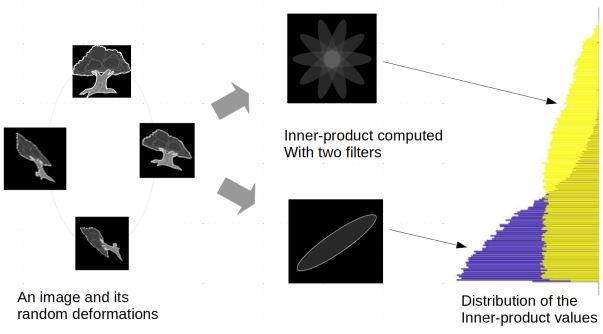


Figure 2: An image I, and two (energy normalized) filters f_1, f_2 are fixed. A sequence of transformations T_i are randomly sampled from $GL_2(\mathbb{R})$ (invertible 2D matrices), to warp the chosen image ($I_i = T_i I$). Plot the histograms of the inner-product values $\langle I_i, f_j \rangle_{j=1,2}$. The complex filter creates a more dispersed distribution (yellow), compared to the simpler one (purple)

This informal notion of simplicity implicitly assumes a set of transformations that remains in the background. With respect to these transformations simplicity means smaller degree of variation. A complex shape is one that experiences wider variations. To formalize further, this transformation set must be specified. Unfortunately, a long-acknowledged hurdle for formalizing a deep learning theory has been the lack of nice analytical models that could describe the variations of natural signals and their distributions. In the absence of such nice external models, we simply look at the variations that are intrinsic in the data.

Definition 2 (Center of a set)**.** Let $\mathbb{S}^{d-1}$ be the unit sphere embedded in $\mathbb{R}^d$, and let $\mathbf{m}$ be a set of points over $\mathbb{S}^{d-1}$ drawn from some (unknown) distribution: $\mathbf{m} = \{x | x \sim \mathbb{S}^{d-1}\}$. The center of $\mathbf{m}$, denoted by $\omega(\mathbf{m})$ is defined as $\omega \in \mathbb{S}^{d-1}$ such that the average distance of the points in $\mathbf{m}$ from ω, given by $\frac{1}{|\mathbf{m}|}\sum_{x \in \mathbf{m}} |x - \omega|^2$ is the minimum.

Definition 3 (Intrinsic Variation Generator)**.** Let $\mathbf{m}$ be a set of points on $\mathbb{S}^{d-1}$ and $\omega(\mathbf{m})$ the center of $\mathbf{m}$ as above. Define *Intrinsic Variation Generator* (IVG) of $\mathbf{m}$ as the set of rotations of $\mathbb{S}^{d-1}$ that can generate every element of $\mathbf{m}$ starting from ω.

$$\mathbb{\Delta}\mathbf{m} = \{\tau | \tau\omega = x \text{ such that } \tau : \mathbb{S}^{d-1} \to \mathbb{S}^{d-1} \text{ and } \quad x \in \mathbf{m}\} \tag{7}$$

If T defines the IVG of some set, we define the adjoint-IVG of T as the collection of usual adjoint operators (for rotations of a sphere, the adjoint coincides with the inverse).

$$\mathbb{H}(T) = \{\tau^{-1} | \tau \in T\} \tag{8}$$

For a set $\mathbf{m} \subseteq \mathbb{S}^{d-1}$, the adjoint-IVG is defined as the adjoint of its IVG.

$$\mathbb{H}\mathbf{m} = \{\tau^{-1} | \tau \in \mathbb{\Delta}\mathbf{m}\} \tag{9}$$

Definition 4 (*Pseudo* Orbit)**.** The *pseudo orbit* (henceforth abbreviated as ***p***-orbit) of a point x with respect to a set of transformation T is the set: $\mathcal{O}_x^T = \{gx | g \in T\}$.

Remark: We do not assume any structure on IVG (or adjoint-IVG). In particular, these sets need not be closed under composition. However, just by definition, IVG and adjoint-IVG of a set $\mathbf{m} \subseteq \mathbb{S}^{d-1}$ are trivially isomorphic: $\mathbb{\Delta}\mathbf{m} \cong \mathbb{H}\mathbf{m}$. So the variation under the action of one reflects the variation under the action of the other. When IVG has additional structures, a ***p***-orbit under its action will also have further structure. For example, if the deformations can be modeled by a compact group, then a ***p***-orbit becomes an orbit of a group-action.

Definition 5 (Dispersion volume of a ***p***-orbit)**.** Let $f \in \mathbb{S}^{d-1}$ be a filter, and its ***p***-orbit under a set of transformations T be denoted by $\mathcal{O}_f^T$ as above. The dispersion is denoted as usual by the covariance matrix:

$$\mathbb{\Sigma}_f^T = \mathbb{E}_{x \sim unif(\mathcal{O}_f^T)}\left[(x - \mathbb{E}[x])(x - \mathbb{E}[x])^T\right] \tag{10}$$

We define the dispersion volume (also referred to as dispersion for simplicity) of the ***p***-orbit of f as the trace of the covariance:

$$\mathbb{V}_f^T = \mathbb{T}\mathbb{r}(\mathbb{\Sigma}_f^T) \tag{11}$$

$\mathbb{V}_f^T$ measures how much variation f goes through under the action of T. In other words, smaller $\mathbb{V}_f^T$ implies higher invariance.

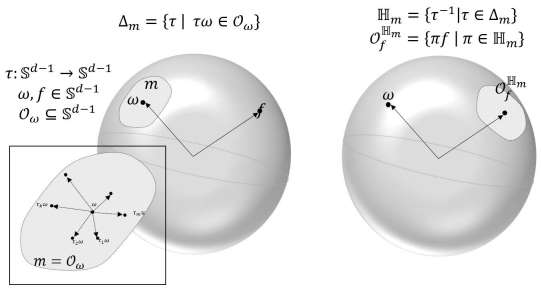


Figure 3: Transfer of probability mass (shown as white patches) from the ***p***-orbit of inputs to the ***p***-orbit of filters. On the left, f is fixed, and inner products are taken with variations of the input. On the right, the center ω is fixed, the other vector is varied over the ***p***-orbit of f under the action of the adjoint-IVG. They are equivalent.

We are now set to analyze our original objective - how contraction biases feature selection. Consider a set of normalized input vectors, $\mathbf{m} = \{x | x \in \mathbb{S}^{d-1}\}$ that is centered on ω and a feature $f \in \mathbb{S}^{d-1}$. For every $\tau \in \mathbb{\Delta}_{\mathbf{m}}$, there is the $\tau^{-1} \in \mathbb{H}_{\mathbf{m}}$ satisfying $\langle \tau\omega, f \rangle = \langle \omega, \tau^{-1} f \rangle$. This is visualized in Figure 3. In effect, the probability mass that was distributed over input-variations, through the inner-product with f, gets redistributed over the ***p***-orbit of f under $\mathbb{H}_{\mathbf{m}}$. It is natural to expect that variance of this distribution must be a function of the dispersion of $\mathcal{O}_f^{\mathbb{H}\mathbf{m}}$, which is formalized in theorem 4.1.

For a fixed set $\mathbf{m}$ of input vectors, and a filter f, we are interested in the spread of the inner-product values. This can be expressed in terms of the IVG, by fixing ω (the center of $\mathbf{m}$) and then varying y over the ***p***-orbit of ω under the action of $\mathbb{\Delta}_{\mathbf{m}}$. Thus we are examining the quantity:

$$Var_{y \sim \mathbf{unif}(\mathcal{O}_\omega^{\mathbb{\Delta}_m})} \Big(\langle f, y \rangle \Big) \tag{12}$$

The variance is a function of ω and $\mathbb{\Delta}_{\mathbf{m}}$. However, if we fixed the IVG, and chose the center uniformly randomly over $\mathbb{S}^{d-1}$, we can get an expectation over the variance. It turns out that this is simply the dispersion of the ***p***-orbit of f under the action of $\mathbb{H}_{\mathbf{m}}$ (modulo a scaling constant). Theorem 4.1 captures this result. To emphasize the fact that the center is a random variable, chosen uniformly randomly over the sphere, we denote it by a new variable u.

Theorem 4.1. *Let T be a set of rotations of the sphere $\mathbb{S}^{d-1}$ and $f \in \mathbb{S}^{d-1}$ be a filter. Assuming T to be the IVG of some set, let $\mathbb{H}(T)$ denote the adjonit-IVG. $\mathbb{V}_f^{\mathbb{H}(T)}$ denotes the dispersion volume of f under $\mathbb{H}(T)$, as defined in definition 5. Then we have*

$$\mathbb{E}_{u \sim \mathbf{unif}(\mathbb{S}^{d-1})} \Big[Var_{y \sim \mathbf{unif}(\mathcal{O}_u^T)} \Big(\langle f, y \rangle \Big) \Big] = \frac{\mathbb{V}_f^{\mathbb{H}(T)}}{d} \tag{13}$$

Assuming $T = \mathbb{\Delta}_{\boldsymbol{m}}$, the dispersion volume becomes $\mathbb{V}_f^{\mathbb{H}\boldsymbol{m}}$.

We prove the theorem in Appendix-B.

4.1 Implications

Let us interpret the consequence of theorem 4.1 . The distribution of the inner-product of a filter and a set of deformations of a random point, is governed by the dispersion volume of the ***p***-orbit of the filter. The smaller the dispersion, the tighter is the distribution. Thus, following theorem 3.1, a filter with smaller dispersion will be preferred by gradient-descent over the ones with larger dispersion. Now, we can connect this back to the original questions we started in section 1. For a filter f to have a small ***p***-orbit means that it has higher invariance to $\mathbb{H}_{\mathbf{m}}$. Noting that $\mathbb{H}_{\mathbf{m}} \cong \mathbb{\Delta}_{\mathbf{m}}$, this really implies that f extracts the essential invariance-information from the inputs (instead of memorizing them). Which is a key ingredient of generalization.

References

Sanjeev Arora, Yingyu Liang, and Tengyu Ma. Why are deep nets reversible: A simple theory, with implications for training. *CoRR*, abs/1511.05653, 2015. URL http://arxiv.org/abs/1511.05653.

Sanjeev Arora, Nadav Cohen, and Elad Hazan. On the optimization of deep networks: Implicit acceleration by overparameterization. *CoRR*, abs/1802.06509, 2018a. URL http://arxiv.org/abs/1802.06509.

Sanjeev Arora, Rong Ge, Behnam Neyshabur, and Yi Zhang. Stronger generalization bounds for deep nets via a compression approach. *CoRR*, abs/1802.05296, 2018b. URL http://arxiv.org/abs/1802.05296.

Peter L Bartlett and Philip M Long. Failures of model-dependent generalization bounds for least-norm interpolation. *J. Mach. Learn. Res.*, 22:204–1, 2021.

Peter L. Bartlett, Andrea Montanari, and Alexander Rakhlin. Deep learning: a statistical viewpoint, 2021. URL https://arxiv.org/abs/2103.09177.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.

Gintare Karolina Dziugaite. *Revisiting generalization for deep learning: PAC-Bayes, flat minima, and generative models*. PhD thesis, University of Cambridge, 2020.

Gintare Karolina Dziugaite and Daniel M Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *arXiv preprint arXiv:1703.11008*, 2017.

Anna C. Gilbert, Yi Zhang, Kibok Lee, Yuting Zhang, and Honglak Lee. Towards understanding the invertibility of convolutional neural networks. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pp. 1703–1710, 2017.

Suriya Gunasekar, Jason D Lee, Daniel Soudry, and Nati Srebro. Implicit bias of gradient descent on linear convolutional networks. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 31*, pp. 9482–9491. Curran Associates, Inc., 2018. URL http://papers.nips.cc/paper/8156-implicit-bias-of-gradient-descent-on-linear-convolutional-networks.pdf.

Moritz Hardt and Tengyu Ma. Identity matters in deep learning. *CoRR*, abs/1611.04231, 2016. URL http://arxiv.org/abs/1611.04231.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL http://arxiv.org/abs/1512.03385.

Renjie Liao, Raquel Urtasun, and Richard Zemel. A pac-bayesian approach to generalization bounds for graph neural networks. *arXiv preprint arXiv:2012.07690*, 2020.

Philip M Long and Hanie Sedghi. Generalization bounds for deep convolutional neural networks. *arXiv preprint arXiv:1905.12600*, 2019.

Michael Naaman. On the tight constant in the multivariate dvoretzky–kiefer–wolfowitz inequality. *Statistics & Probability Letters*, 173:109088, 2021. ISSN 0167-7152. doi: https://doi.org/10.1016/j.spl.2021.109088. URL https://www.sciencedirect.com/science/article/pii/S016771522100050X.

Vaishnavh Nagarajan. Explaining generalization in deep learning: progress and fundamental limits. *arXiv preprint arXiv:2110.08922*, 2021.

Vaishnavh Nagarajan and J. Zico Kolter. Uniform convergence may be unable to explain generalization in deep learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019a. URL https://proceedings.neurips.cc/paper/2019/file/05e97c207235d63ceb1db43c60db7bbb-Paper.pdf.

Vaishnavh Nagarajan and Zico Kolter. Deterministic PAC-bayesian generalization bounds for deep networks via generalizing noise-resilience. In *International Conference on Learning Representations*, 2019b. URL https://openreview.net/forum?id=Hygn2o0qKX.

Behnam Neyshabur, Zhiyuan Li, Srinadh Bhojanapalli, Yann LeCun, and Nathan Srebro. Towards understanding the role of over-parametrization in generalization of neural networks. *CoRR*, abs/1805.12076, 2018. URL http://arxiv.org/abs/1805.12076.

Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.

Ankit B Patel, Minh Tan Nguyen, and Richard Baraniuk. A probabilistic framework for deep learning. In D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garne tt (eds.), *Advances in Neural Information Processing Systems 29*, pp. 2558–2566. Curran Associates, Inc., 2016. URL http://papers.nips.cc/paper/6231-a-probabilistic-framework-for-deep-lear\ning.pdf.

Arnab Paul and Suresh Venkatasubramanian. Why does deep learning work? - A perspective from group theory. *CoRR*, abs/1412.6621, 2014. URL http://arxiv.org/abs/1412.6621.

Tomaso A. Poggio, Kenji Kawaguchi, Qianli Liao, Brando Miranda, Lorenzo Rosasco, Xavier Boix, Jack Hidary, and Hrushikesh Mhaskar. Theory of deep learning III: explaining the non-overfitting puzzle. *CoRR*, abs/1801.00173, 2018. URL http://arxiv.org/abs/1801.00173.

Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dicksteiñ, and Surya Ganguli. Exponential expressivity in deep neural networks through transient cha os. In D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garne tt (eds.), *Advances in Neural Information Processing Systems 29*, pp. 3360–3368. Curran Associates, Inc., 2016. URL http://papers.nips.cc/paper/6322-exponential-expressivity-in-deep-neural\-networks-through-transient-chaos.pdf.

S. S. Schoenholz, J. Gilmer, S. Ganguli, and J. Sohl-Dickstein. Deep Information Propagation. *ArXiv e-prints*, November 2016.

Daniel Soudry, Elad Hoffer, and Nathan Srebro. The implicit bias of gradient descent on separable data. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=r1q7n9gAb.

Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. *CoRR*, abs/1503.02406, 2015. URL http://arxiv.org/abs/1503.02406.

René Vidal, Joan Bruna, Raja Giryes, and Stefano Soatto. Mathematics of deep learning. *CoRR*, abs/1712.04741, 2017. URL http://arxiv.org/abs/1712.04741.

M.J. Wainwright. *High-Dimensional Statistics: A Non-Asymptotic Viewpoint*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 2019. ISBN 9781108498029. URL https://books.google.com/books?id=8C8nuQEACAAJ.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *International Conference on Learning Representations (ICLR)*, abs/1611.03530, 2017. URL http://arxiv.org/abs/1611.03530.

Chiyuan Zhang, Kushal Tirumala, and Jagath Vytheeswaran. Simple demo code for training over-parameterized models on cifar-10 dataset. https://github.com/pluskid/fitting-random-labels, 2020.

Wenda Zhou, Victor Veitch, Morgane Austern, Ryan P Adams, and Peter Orbanz. Non-vacuous generalization bounds at the imagenet scale: a pac-bayesian compression approach. *arXiv preprint arXiv:1804.05862*, 2018.

SUPPLEMENTAL INFORMATION

A APPENDIX A1

To prove $\mathbf{GC}(\epsilon, \delta)$-reliability of a linear transformation (lemma 2.2), we establish the same for operational building blocks (lemma A.1 - A.4).

Lemma A.1. *[Survival function] If $X \in \mathbb{R}^d$ is $\mathbf{GC}(\epsilon, \delta)$-reliable then the survival function of X is also close to its empirical estimate,* i.e.*, $|\mathbf{S}^X(x) - \mathbf{S}_n^X(x)| < \epsilon$, for all x with probability at least $1 - \delta$.*

Proof. This directly follows from the definition of the survival function. Simply observe that $\mathbf{S}^X(x) = \mathbf{Pr}(X > x) = 1 - \mathbf{F}^X(x)$. So, $|\mathbf{S}^X(x) - \mathbf{S}_n^X(x)| = |\mathbf{F}^X(x) - \mathbf{F}_n^X(x)| \leq \epsilon$ with probability at least $1 - \delta$. □

Lemma A.2. *[Negation] If $X \in \mathbb{R}^d$ is $\mathbf{GC}(\epsilon, \delta)$-reliable then so is the negated variable $-X$.*

Proof. $\mathbf{F}^{-X}(t) = \mathbf{Pr}(-X \leq t) = \mathbf{Pr}(X \geq -t) = \mathbf{S}^X(-t)$. Thus $|\mathbf{F}^{-X}(t) - \mathbf{F}_n^{-X}(t)| = |\mathbf{S}^X(-t) - \mathbf{S}_n^X(-t)| \leq \epsilon$, for all t (by lemma A.1). □

Lemma A.3. *[Scaling] If X is $\mathbf{GC}(\epsilon, \delta)$-reliable, and $a \in \mathbb{R}$, then aX is also $\mathbf{GC}(\epsilon, \delta)$-reliable.*

Proof. This is easy to see for $a > 0$. Let $X_a = aX$. Observe that $\mathbf{F}^{X_a}(t) = \mathbf{Pr}(X_a \leq t) = \mathbf{Pr}(X \leq t/a) = \mathbf{F}^X(t/a)$. Hence $\mathbf{GC}(\epsilon, \delta)$-reliability of X_a immediately follows.

For $a < 0$, observe that $X_{-a} = -aX$ must be $\mathbf{GC}(\epsilon, \delta)$-reliable by the above argument. But since $X_a = -X_{-a}$, by lemma A.2, X_a must also be $\mathbf{GC}(\epsilon, \delta)$-reliable. □

Lemma A.4. *[Addition] If $X, Y \in \mathbb{R}$ are $\mathbf{GC}(\epsilon, \delta)$-reliable, then $Z \coloneqq X + Y$ is also $\mathbf{GC}(\epsilon, \delta)$-reliable.*

Proof. We start by observing that when individual coordinates X and Y are $\mathbf{GC}(\epsilon, \delta)$-reliable, the variable representing the Cartesian-product space, say $U \coloneqq (X, Y)$, is also $\mathbf{GC}(\epsilon, \delta)$-reliable. This follows directly from the definition of a multivariate CDF as given in Theorem 2.1.

For $t \in \mathbb{R}$, define line $\mathbf{L}_t \coloneqq \{(x, y) \in \mathbb{R}^2 | x + y = t\}$, and the half-space $\mathcal{H}_t \coloneqq \{(x, y) \in \mathbb{R}^2 | x + y \leq t\}$. For any point $p \in \mathbb{R}^2$, also denote the rectangular (unbounded) region $\mathcal{R}_p \coloneqq \{q \in \mathbb{R}^2 | q \leq p\}$. Also observe that

$$\mathcal{H}_t = \bigcup_{p \in \mathbf{L}_t} \mathcal{R}_p \tag{14}$$

We can then decompose the indicator function over $\mathcal{H}_t$ using indicators of sets $\mathcal{R}_p$ as follows:

$$\begin{aligned} & \mathbb{1}_{((X,Y) \in \mathcal{H}_t)} \\ =& \mathbb{1}_{((X,Y) \in \bigcup\limits_{p \in \mathbf{L}_t} \mathcal{R}_p)} && \text{directly from equation 14} \\ =& \sup_{p \in \mathbf{L}_t} \{\mathbb{1}_{((X,Y) \in \mathcal{R}_p)}\} && \text{since } \mathbb{1}_{(A \cup B)} = \max\{\mathbb{1}_A, \mathbb{1}_B\} \end{aligned} \tag{15}$$

For $t \in \mathbb{R}$, we can now express the CDF of Z in terms of sets $\mathcal{R}_p$ over the range of $U = (X, Y)$:

$$\begin{aligned} \mathbf{F}^Z(t) &= \mathbf{Pr}[Z \leq t] \\ &= \mathbf{Pr}[(X, Y) \in \mathcal{H}_t] \\ &= \mathbb{E}[\mathbb{1}_{((X,Y) \in \mathcal{H}_t)}] \\ &= \mathbb{E}\left[\sup_{p \in \mathbf{L}_t} \{\mathbb{1}_{((X,Y) \in \mathcal{R}_p)}\}\right] \end{aligned} \tag{16}$$

This means that the empirical CDF will have the form :

$$\mathbf{F}_n^Z(t) = \frac{1}{n} \sum_{j=1}^{n} \sup_{p \in \mathbf{L}_t} \{\mathbb{1}_{((x^{(j)}, y^{(j)}) \in \mathcal{R}_p)}\} \tag{17}$$

If Z ***was not*** $\mathbf{GC}(\epsilon, \delta)$-reliable, then there would be some $t \in \mathbb{R}$, for which $\mathbf{F}^Z(t)$ and $\mathbf{F}_n^Z(t)$ would not be close with high probability. In terms of decomposition of these CDFs as given in equations 16 and 17, then there must exist some $\mathcal{R}_p$ for which $\mathbb{E}[\mathbb{1}_{u \in \mathcal{R}_p}]$ and its empirical version are not close.

However, as noted in the beginning of this proof, the variable $U = (X, Y)$ is $\mathbf{GC}(\epsilon, \delta)$-reliable, and thus, for all $p \in \mathbb{R}^2$, we have that $|\mathbb{E}[\mathbb{1}_{(u \in \mathcal{R}_p)}] - \frac{1}{n} \sum_j \mathbb{1}_{(u^{(j)} \in \mathcal{R}_p)}| \leq \epsilon$ holds with probability at least $1 - \delta$. Thus $Z = X + Y$ must be $\mathbf{GC}(\epsilon, \delta)$-reliable.

□

Now we are ready to prove lemma 2.2.

Lemma 2.2 : If X_{l-1} is $\mathbf{GC}(\epsilon, \delta)$-reliable, then $Z = W_l X_{l-1}$ is also $\mathbf{GC}(\epsilon, \delta)$-reliable.

Proof. Since the context for this is any layer l, to make notation simpler, we drop the subscript l, for the remainder of this proof, *i.e.*, we set $X \coloneqq X_{l-1}, W \coloneqq W_{l-1}$ and thus $Z = WX$. Let the dimensions of the spaces be given as follows: $X \in \mathbb{R}^d, Z \in \mathbb{R}^{d'}$ and $W \in \mathbb{R}^{(d \times d')}$.

Let $W = [\theta_1, \theta_2, \ldots \theta_{d'}]$, where each $\theta_k \in \mathbb{R}^d$. We will prove GC-reliability for each coordinate $Z_k = {\theta_k}^T X$ for $1 \leq k \leq d'$.

Next, fixing k, we can further simplify notation by getting rid of subscript k : define $\beta \coloneqq \theta_k$ and $Y \coloneqq Z_k$. Our first goal then is to show that Y is $\mathbf{GC}(\epsilon, \delta)$-reliable.

So, we have $Y = \sum_{j=1}^d \beta_j X_j$, where $\beta_j, X_j \in \mathbb{R}$, and for all j, X_j is $\mathbf{GC}(\epsilon, \delta)$-reliable. Define $Q_j \coloneqq \beta_j X_j$, and by lemma A.3, we have Q_j is $\mathbf{GC}(\epsilon, \delta)$-reliable.

Next, for integer $1 \leq r \leq d$, define $Q_{[1:r]} \coloneqq \sum_{j=1}^r Q_j$. Thus we have a recursive expression $Q_{[1:r]} = Q_{[1:r-1]} + Q_r$, and also the fact $Y = Q_{[1:d]}$. Since, every Q_j is $\mathbf{GC}(\epsilon, \delta)$-reliable, using lemma A.4, and starting from $Q_{[1:2]} = Q_1 + Q_2$, we can inductively claim that $Q_{[1:r]}$ is $\mathbf{GC}(\epsilon, \delta)$-reliable for all $r \leq d$. Therefore Y is $\mathbf{GC}(\epsilon, \delta)$-reliable.

Since $Y = Z_k$, we have established that the variable Z is $\mathbf{GC}(\epsilon, \delta)$-reliable on every coordinate $1 \leq k \leq d'$. Thus Z must be $\mathbf{GC}(\epsilon, \delta)$-reliable.

□

Lemma 2.4 : Let $Y \in \mathbb{R}^M$ be the last-layer's output restricted to input samples for a fixed class c. and $\zeta(Y) \in \mathbb{R}^M$ be the softmax function, *i.e.*, for $k \in \{1, 2 \ldots M\}$, $\zeta_k = \frac{\exp(Y_k)}{\sum_{j=1}^M \exp(Y_j)}$ represents the probability that a sample should be classified as class k. If Y is $\mathbf{GC}(\epsilon, \delta)$-reliable, then ζ_c is $\mathbf{GC}(M\epsilon, \delta)$-reliable.

Proof. Observe that in order for a sample to be adjudicated as a member of class c, it must be the case that $Y_c > Y_{c'}$, for all $c' \neq c$. $\mathbf{GC}(\epsilon, \delta)$-reliability of Y only asserts that with probability at least $1 - \delta$, $|\mathbf{S}^{Y_{c'}}(y) - \mathbf{S}_n^{Y_{c'}}(y)| \leq \epsilon$, for all $y \in \mathbb{R}$ (using lemma A.1). In particular, setting $y = Y_c$, we need to allow for $\mathbf{S}^{Y_{c'}}(Y_c) \leq \mathbf{S}_n^{Y_{c'}}(Y_c) + \epsilon$. That means, even if the training error is 0.0%, the bound still allows for up to an ϵ fraction of samples of class c getting wrongly classified as class c'.

We define the following variables. Let $Z_k \coloneqq \exp(Y_k)$. Clearly Z_k is also $\mathbf{GC}(\epsilon, \delta)$-reliable for every k. Also for any class c', let $t_{c'} \coloneqq \sum_{k \neq c'} z_k$, where z_k denotes a specific value taken by the r.v Z_k. Finally for $t > 0$, let $r \coloneqq z_c/t - t_{c'}$. By $\mathbf{GC}(\epsilon, \delta)$-reliability of $Z_{c'}$, the following holds :

$$\mathbf{S}^{Z_{c'}}(r) \quad \leq \quad \mathbf{S}_n^{Z_{c'}}(r) + \epsilon \text{ holds for all } c' \tag{18}$$

By definition, $\zeta_c = \frac{Z_c}{\sum_j Z_j}$. Therefore, for any fixed set of values $\{z_k | k \neq c'\}$, setting $z_{c'} = r$, we get $\zeta_c = t$. Equation 18 above suggests that $\mathbf{Pr}(Z_{c'}) \geq r$ can have a deviation of ϵ from the empirical estimate. But, assuming every z_k ($k \neq c'$) is fixed, if $Z_{c'} \geq r$, we will get $\zeta_c \leq t$. In other words, the ϵ uncertainty incurred by the class c' in equation 18, could add up to (and no more than) ϵ variation to the empirical CDF of ζ_c. That means $\mathbf{F}^{\zeta_c}(t) \leq \mathbf{F}_n^{\zeta_c}(t) + \epsilon$, considering the uncertainty

added solely by class c'. Since the classes are mutually exclusive, and for any c' the deviation from its empirical estimate is bounded by ϵ, we conclude that: $|\mathbf{F}^{\zeta_c}(t) - \mathbf{F}_n^{\zeta_c}(t)| \leq M\epsilon$ for all $t \in \mathbb{R}$ with probability at least $1-\delta$. Hence, ζ_c is $\mathbf{GC}(M\epsilon, \delta)$-reliable. □

Lemma 2.3 If Y is $\mathbf{GC}(\epsilon, \delta)$-reliable, then so is $\sigma(Y)$.

Proof. Let $Z = \sigma(Y)$. Since Y is $\mathbf{GC}(\epsilon, \delta)$-reliable, we have $|\mathbf{F}^Y(0) - \mathbf{F}_n^Y(0)| \leq \epsilon$ with probability at least $1-\delta$. We also have, by the very definition of ReLU, $F^Z(0) = F^Y(0)$. Therefore, $|\mathbf{F}^Z(0) - \mathbf{F}_n^Z(0)| \leq \epsilon]$ with probability at least $1-\delta$. For $Y \geq 0$ we have $Z = Y$, and therefore, $\forall t, \mathbf{F}_n^Z(t) = \mathbf{F}_n^Y(t)$. Finally, for all $t \leq 0$, $\mathbf{F}^Z(t) = \mathbf{F}_n^Z(t) = 0$ is trivially true. Hence the result follows. □

Theorem 2.5 : Let $Y = h(X)$ be a neural net that learns a M-way classification task. Given a small positive constant κ_0, let S_{κ_0} be a κ_0-filtered set of training samples drawn *i.i.d* from some data-distribution $\mathcal{D}_{x|c}^n$ (as in equation 5, so that they all have fixed class label c) , *i.e.*, $S_{\kappa_0} \sim \mathcal{D}_{x|c}^n(\mathbb{R}^d)$. Let $\ell^c(y)$ be the loss function evaluated on the coordinate Y_c, that satisfies the following properties: (*i*) $\ell^c(y)$ is only finitely supported and (*ii*) over this finite support, $\frac{d}{dy}[\ell^c(y)] < 0$. Then, with probability at least $1-\delta$ the generalization error $\epsilon_{\texttt{gen}}$ is upper-bounded as follows:

$$\epsilon_{\texttt{gen}} \leq M\left[|\max\{\ell^c\} - \min\{\ell^c\}|\right]\sqrt{\frac{1}{2n}\log\frac{n(d+1)}{\delta}} \tag{19}$$

We first prove a slightly simpler result, over a function $g(x)$ that mimics the loss function $\ell^c()$ in this theorem.

Lemma A.5. *Let X be a real valued non-negative random variable satisfying the condition that its empirical CDFs are close,* i.e.*there exist $\epsilon \geq 0$ and $1 \geq \delta \geq 0$ such that $|\mathbf{F}_n^X(t) - \mathbf{F}^X(t)| \leq \epsilon$ with probability at least $(1-\delta)$. Let $g(X) : \mathbb{R}_+ \to \mathbb{R}$, be a finitely supported function such that $g(t) = g'(t) = 0$ for $t \geq B$, and $g'(t) \leq 0$ for $t \geq 0$. Then, the difference between the empirical and true means of g is upper-bounded as follows:*

$$\left|\mathbb{E}_n g - \mathbb{E}g\right| \leq \epsilon[g(0) - g(B)] \tag{20}$$

Proof. Let X be a random variable taking values in $\mathbb{R}_+$. The following alternative formulation of expectation of a function is well known.

$$\begin{aligned}
g(X) &= \int_0^X g'(t)dt + g(0) \\
\implies g(X) &= \int_0^\infty \mathbb{1}_{(t \leq X)} g'(t)dt + g(0) \\
\implies \mathbb{E}[g(X)] &= \int_0^\infty \mathbb{E}[\mathbb{1}_{(t \leq X)}] g'(t)dt + g(0) \\
\implies \mathbb{E}[g(X)] &= \int_0^\infty (1 - \mathbf{F}^X(t)) g'(t)dt + g(0)
\end{aligned} \tag{21}$$

Plugging in the empirical CDF in equation 21, we can get an alternative formulation for the empirical mean

$$\mathbb{E}_n g = \int_0^\infty (1 - \mathbf{F}_n^X(t)) g'(t)dt + g(0) \tag{22}$$

So, comparing the true and empirical means

$$\begin{aligned}
&\left|\mathbb{E}_n g - \mathbb{E} g\right| \\
&= \left| \int_0^\infty (\mathbf{F}_n^X(t) - \mathbf{F}^X(t)) g'(t) dt \right| \\
&\leq \int_0^\infty \left|\mathbf{F}_n^X(t) - \mathbf{F}^X(t)\right| . \left| g'(t) \right| dt \\
&\leq \epsilon \int_0^\infty \left| g'(t) \right| dt && \text{assumption made in lemma A.5} \\
&\leq -\epsilon \int_0^B g'(t) dt && \text{since } |g'(t)| = -g'(t) \\
&= \epsilon [g(0) - g(B)]
\end{aligned} \tag{23}$$

□

Now we can prove theorem 2.5 as follows.

Proof of Theorem 2.5. This is done in stages. First we obtain a bound on the CDF of input samples via the DKW-theorem 2.1 : $\epsilon = \sqrt{\frac{1}{2n} \log \frac{n(d+1)}{\delta}}$ Next, observe that since our neural net has a softmax layer at the end, the bound really becomes $M\epsilon$. Then, we can substitute function $\ell^c(y)$ for $g()$, and $M\epsilon$ for ϵ in lemma A.5, which yields : $|\mathbb{E}[\ell^c] - \mathbb{E}_n[\ell^c]| \leq M\epsilon |\max\{\ell^c\} - \min\{\ell^c\}|$. Realizing that the left hand side of this inequality is actually $\epsilon_{\texttt{gen}}$, and finally substituting $\epsilon = \sqrt{\frac{1}{2n} \log \frac{n(d+1)}{\delta}}$, gives the desired result.

□

APPENDIX A2

In this section, we will prove **Theorem 3.1**. We set up some required notation, and prove preliminary results.

Let θ, σ be the linear and ReLU units of a neural layer such that $V = \theta^T U$ and W = $\sigma(V)$. Let the back-propagated error-vectors at each stage be denoted by e_w, e_v, e_u correspondingly. The back-propagation step dictates the following relationship between the error-vectors: $e_v = e_w \odot \sigma'(V)$ and $e_u = e_v^T \theta$. The symbol $\odot$ denotes point-wise multiplication.

Pictorially, their flows look as follows:

$$\begin{array}{lll}
U & \xrightarrow{\theta} V = \theta^T U & \xrightarrow{\sigma} W = \sigma(V) \\
e_u = e_v^T \theta & \leftarrow e_v = e_w \odot \sigma'(v) & \leftarrow e_w
\end{array}$$

Note that that saying $Var(W(t+1)) < Var(W(t))$ is equivalent to saying $Var((W - e_w)) < Var(W)$. So, starting from here, we get the following identity.

$$\begin{aligned}
& Var(W - e_w) \leq && Var(W) \\
\implies \quad & Var(W) + Var(e_w) - 2cov(W, e_w) \leq && Var(W) \\
\implies \quad & cov(W, e_w) \geq && \frac{1}{2} Var(e) \\
\implies \quad & cov(W, e_w) \geq 0
\end{aligned} \tag{24}$$

If we are restricted to only samples that are advancing through a layer, then we have that $W = V$ and $\sigma'(V) = 1$. Which means $e_v = e_w \odot \sigma'(V) = e_w$. Thus

$$cov((V, e_v)) = cov((W, e_w)) > 0 \tag{25}$$

Finally, we look at one step of SGD adjustment, which generally looks like the following

$$\theta(t+1) \coloneqq \theta(t) - \eta \nabla_\theta(\mathcal{L})$$

However, as the error vector is back-propagated, the term $\eta \nabla_\theta(\mathcal{L})$ essentially reduces to:

$$\eta \nabla_\theta(\mathcal{L}) = \eta e_v U^T \tag{26}$$

Next we prove the following lemma that shows that adjusting the weights θ proportionately to the outer-product ($e_v U^T$) essentially reduces dispersion of V.

Lemma A.6. *Let e_w be the back-propagated error-vector at the layer of W such that the adjusted dispersion volume of W is reduced from before* i.e.*, $Var(W - e_w) \leq Var(W)$. Assume that the learning rate η be a small number such that $\mathcal{O}(\eta^2) \approx 0$, and the change in the weight parameter θ as per SGD is given by $\theta \leftarrow [\theta - \eta \nabla_\theta(\mathcal{L})]$, then the resulting dispersion volume of $V = \theta^T U$ decreases after the SGD adjustment.*

$$Var(W - e_w) \leq Var(W) \implies Var((\theta - \eta \nabla_\theta(\mathcal{L}))^T U) \leq Var(\theta^T U) \tag{27}$$

Proof.

$$\begin{aligned} &Var((\theta - \eta \nabla_\theta(\mathcal{L}))^T U) \\ &= Var(\theta^T U) + \eta^2 Var(\nabla_\theta^T(\mathcal{L}) U) - 2\eta cov(\theta^T U, (\nabla_\theta^T(\mathcal{L}) U)) \\ &= Var(\theta^T U) - 2\eta cov(\theta^T U, (\nabla_\theta^T(\mathcal{L}) U)) \end{aligned} \tag{28}$$

So, the difference in the dispersion after the learning-adjustment step becomes

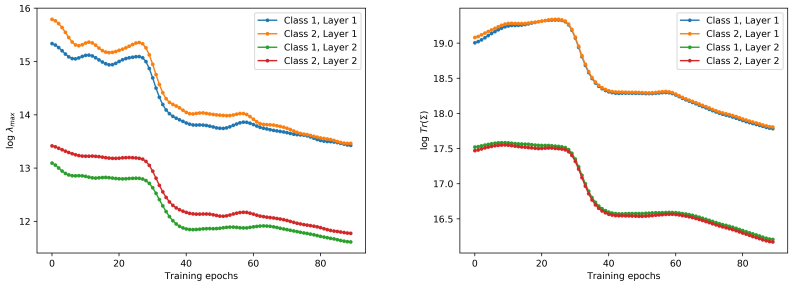
$$\begin{aligned} &Var(\theta^T U) - Var((\theta - \eta \nabla_\theta(\mathcal{L}))^T U) \\ &= 2\eta cov(\theta^T U, \nabla_\theta^T(\mathcal{L}) U) \\ &= 2\eta cov(V, [e_w \odot \sigma'(V) U^T] U) && \text{using } e_v = e_w \odot \sigma'(V) \\ &= 2\eta cov(V, [e_w . U^T U]) && \sigma'(V) = 1 \text{ for advancing samples} \\ &= 2\eta cov(W, |U|^2 e_w) \\ &= 2\eta |U|^2 cov(W, e_w) && cov(W, e_w) \geq 0 \text{ from eqn 24} \\ &\geq 0 \end{aligned} \tag{29}$$

□

Finally we are ready to prove theorem 3.1

Theorem 3.1 During gradient-descent, if the output probability mass contracts at time-step t, *i.e.*, $Var(X_L(t+1)) < Var(X_L(t))$, then this contraction proceeds to inner layers. In other words, even for $1 < l < L$ we get $Var(X_l(t+1)) < Var(X_l(t))$.

Proof. This follows by repeatedly applying the results of lemma A.6 to layers $L - 1 \geq l \geq 2$. At every step X_l plays the role of V(in lemma A.6). □



(a) Evolution of $\lambda_{max}(t)$ (b) Evolution of $\mathbb{Tr}(\mathbb{\Sigma}_{c,l}(t))$

Figure 4: Evolution of various dispersion parameters as functions of training epochs

APPENDIX A3

We describe a simple experiment that corroborates the prediction of theorem 3.1. We trained a neural net over imagenet classification data set Deng et al. (2009), which has 1000 classes and 1.3 million images. We used the resnet-18 He et al. (2015) model that comes with pytorch distribution Paszke et al. (2017). The network was randomly initialized, and trained for 90 epochs over the entire datatset, with cross-entropy loss (default settings). A network snapshot was taken after every epoch.

Post-training, we randomly chose two classes (marked Class 1 and 2 in the figures). Their input instances were forward-passed through every snapshot, and we collected the outputs at two layers - layer-1, and layer-2 (pytorch convention for layer names). Each class has 1300 samples, resulting in a cloud of 1300 multidimensional points at each layer. Their dispersion statistics clearly suggest the occurrence of contraction, as we discuss below.

Let $\mathbf{m}_l^c(t)$ denote the point-cloud obtained for class c at layer l. For this discussion $c \in \{1, 2\}$ and $l \in \{1, 2\}$. Thus $\mathbf{m}_l^c(t) = \{x | x \sim \mu_l^c(t)\}$. Let $\mathbb{\Sigma}_{c,l}(t)$ be the covariance matrix of $\mathbf{m}_l^c(t)$. We look at the eigenvalues of $\mathbb{\Sigma}_{c,l}(t)$. Figure 4a plots the evolution the maximum eigenvalue as a function of the training epoch t. For both classes, at both layers, the maximum eigenvalue shows a clear downtrend along training epochs. Figure 4b plots the matrix-trace of the same, which also follows a downtrend suggesting shrinkage in all directions.

APPENDIX B

Theorem 4.1 : *Let T be a set of rotations of the sphere $\mathbb{S}^{d-1}$ and $f \in \mathbb{S}^{d-1}$ be a filter. Assuming T to be the IVG of some set, let $\mathbb{H}(T)$ denote the adjonit-IVG. $\mathbb{V}_f^{\mathbb{H}(T)}$ denotes the dispersion volume of f under $\mathbb{H}(T)$, as defined in definition 5. Then we have*

$$\mathbb{E}_{u \sim \mathbf{unif}(\mathbb{S}^{d-1})} \left[Var_{x \sim \mathbf{unif}(\mathcal{O}_u^T)} \left(\langle f, x \rangle \right) \right] = \frac{\mathbb{V}_f^{\mathbb{H}(T)}}{d} \tag{30}$$

We first need to prove an auxiliary result that we will use in the actual proof.

Lemma A.7. *Let $\lambda_1, \lambda_2, \ldots \lambda_d \in \mathbb{R}$, and $u \sim unif(\mathbb{S}^{d-1})$. Then*

$$\mathbb{E}_{u \sim unif(\mathbb{S}^{d-1})} \left[\sum_j (\alpha_j^u)^2 \lambda_j \right] = \frac{\sum_j \lambda_j}{d} \tag{31}$$

where $\alpha_{j=1\ldots d}^u \in \mathbb{R}$ are the coefficients of u along the basis vectors.

Proof: Since u is uniformly sampled across $\mathbb{S}^{d-1}$, we can approximate that by sampling n points $\alpha^{(1)}, \alpha^{(2)} \dots \alpha^{(n)} \in \mathbb{S}^{d-1}$ uniformly:

$$\mathbb{E}_{u \sim unif(\mathbb{S}^{d-1})}\left[\sum_j (\alpha_j^u)^2 \lambda_j\right] = \lim_{n\to\infty} \frac{1}{n} \sum_i \sum_j \lambda_j (\alpha_j^{(i)})^2 \tag{32}$$

Each $\alpha^{(i)} \in \mathbb{S}^{d-1}$, so, we have the following:

$$\begin{aligned} (\alpha_1^{(1)})^2 + (\alpha_2^{(1)})^2 + \ldots + (\alpha_d^{(1)})^2 &= 1 \\ (\alpha_1^{(2)})^2 + (\alpha_2^{(2)})^2 + \ldots + (\alpha_d^{(2)})^2 &= 1 \\ &\vdots \\ (\alpha_1^{(n)})^2 + (\alpha_2^{(n)})^2 + \ldots + (\alpha_d^{(n)})^2 &= 1 \end{aligned} \tag{33}$$

However, the points $\alpha^{(i)}$ also uniformly sample the unit sphere. Thus, the rotational symmetry dictates that each column in equation 33 must add up to the same, resulting in

$$\begin{aligned} & \sum_i (\alpha_j^{(i)})^2 = \frac{n}{d} \\ \implies & \sum_i \lambda_j (\alpha_j^{(i)})^2 = \frac{n}{d} \lambda_j \\ \implies & \sum_j \sum_i \lambda_j (\alpha_j^{(i)})^2 = \sum_j \frac{n}{d} \lambda_j \\ \implies & \frac{1}{n} \sum_j \sum_i \lambda_j (\alpha_j^{(i)})^2 = \frac{\sum_j \lambda_j}{d} \\ \implies & \mathbb{E}_{u \sim unif(\mathbb{S}^{d-1})}\left[\sum_j (\alpha_j^u)^2 \lambda_j\right] = \frac{\sum_j \lambda_j}{d} \quad \square \end{aligned} \tag{34}$$

Now we are ready to prove theorem 4.1. First, fix a vector (think of it as a possible input to a layer) $u \in \mathbb{S}^{d-1}$. Let the actual input vector be chosen from the uniform distribution over the $\boldsymbol{p}$-orbit of u under the action of T. Let f be one filter of the network. We examine the variance of the inner-product $\langle f, x \rangle$ as x varies over the $\boldsymbol{p}$-orbit $\mathcal{O}_u^T$. Let us denote the variance as $Var_{x \sim \mathbf{unif}(\mathcal{O}_u)}(\langle f, x \rangle)$.

However, for every $g \in T$, there is a $g^{-1} \in \mathbb{H}(T)$ such that $\langle gu, f \rangle = \langle u, g^{-1} f \rangle$. Thus the probability-mass over $\mathcal{O}_u^T$ simply transfers to a probability-mass over $\mathcal{O}_f^{\mathbb{H}(T)}$. Moreover, since the probability mass is transferred through a simple dot-product, a uniform distribution will result in another uniform distribution. Thus

$$Var_{x \sim \mathcal{O}_u^T}(\langle f, x \rangle) = Var_{v \sim \mathcal{O}_f^{\mathbb{H}(T)}}(\langle u, v \rangle) \tag{35}$$

For the rest of the proof, we will simplify our notations. Since it is understood that x in equation 35 always varies under the action of T, and v will vary only under the action of $\mathbb{H}(T)$, we will drop the superscript by using the following simplification.

$$\mathcal{O}_u := \mathcal{O}_u^T \qquad \text{and} \qquad \mathcal{O}_f := \mathcal{O}_f^{\mathbb{H}(T)} \tag{36}$$

Denote the average of the $\boldsymbol{p}$-orbit $\mathcal{O}_f$ as

$$\boldsymbol{\mu}_f = \mathbb{E}_{v \sim \mathbf{unif}(\mathcal{O}_f)}[v] \tag{37}$$

Now, we examine the RHS of equation 35.

$$
\begin{aligned}
&Var_{v\sim\mathbf{unif}(\mathcal{O}_f)}(\langle u, v\rangle) \\
&= \mathbb{E}_{v\sim\mathbf{unif}(\mathcal{O}_f)}[(u^T(v-\boldsymbol{\mu}_f))(u^T(v-\boldsymbol{\mu}_f))^T] \\
&= \mathbb{E}_{v\sim\mathbf{unif}(\mathcal{O}_f)}[(u^T(v-\boldsymbol{\mu}_f))((v-\boldsymbol{\mu}_f)^T u)] \\
&= u^T\left\{\mathbb{E}_{v\sim\mathbf{unif}(\mathcal{O}_f)}[(v-\boldsymbol{\mu}_f)(v-\boldsymbol{\mu}_f)^T]\right\}u \\
&= u^T\mathbb{\Sigma}_f u \qquad \text{recalling the definition of } \mathbb{\Sigma}_f \text{from equation 10}
\end{aligned}
\tag{38}
$$

In the above, we set $\mathbb{\Sigma}_f := \mathbb{\Sigma}_f^{\mathbb{H}(T)}$ for keeping notations simple.
$\mathbb{\Sigma}_f$ is positive-semidefinite, and thus its eigenvectors ξ_j form an orthogonal basis of $\mathbb{R}^d$. Let the corresponding eigenvalues are λ_j. Expanding u in that basis, we get:

$$
\begin{aligned}
&Var_{v\sim\mathbf{unif}(\mathcal{O}_f)}(\langle u, v\rangle) \\
&= u^T\mathbb{\Sigma}_f u \qquad\qquad\qquad \text{from equation 38} \\
&= \left[\sum_j \alpha_j^{(u)}\xi_j\right]^T \mathbb{\Sigma}_f \left[\sum_k \alpha_k^{(u)}\xi_k\right] \qquad \alpha_j^{(u)} \text{are the coefficients in the eigenbasis} \\
&= \left[\sum_k \alpha_j^{(u)}\xi_j\right]^T \left[\sum_j \alpha_k^{(u)}\lambda_k\xi_j\right] \qquad \text{since } \mathbb{\Sigma}_f\xi_k = \lambda_k\xi_k \\
&= \sum_j\sum_k \alpha_j^{(u)}\alpha_k^{(u)}\lambda_k\xi_j^T\xi_k \\
&= \sum_j \lambda_j(\alpha_j^{(u)})^2\xi_j^T\xi_j \quad \text{for } j\neq k, \text{eigenvectors are orthogonal } (\xi_j^T\xi_k = 0) \\
&= \sum_j \lambda_j(\alpha_j^{(u)})^2 \quad \text{since } \xi_j^T \text{ are unit vectors, } \xi_j^T\xi_j = 1
\end{aligned}
\tag{39}
$$

Since $u \in \mathbb{S}^{d-1}$ and therefore $\sum_j(\alpha_j^{(u)})^2 = 1$. Note that so far we held u constant, while varying over its $\boldsymbol{p}$-orbit $\mathcal{O}_u$ (which then transferred the probability mass to the $\boldsymbol{p}$-orbit of f). Equation 38 computes the variance of this density in terms of the covariance of $\mathcal{O}_f$. We can now estimate the expectation of this variance if we choose u uniformly randomly from $\mathbb{S}^{d-1}$. That means we want to evaluate the quantity

$$
\mathbb{E}_{u\sim\mathbf{unif}(\mathbb{S}^{d-1})}\left[Var_{x\sim\mathbf{unif}(\mathcal{O}_u)}(\langle f, x\rangle)\right]
$$

Expanding this expected variance in terms of the quantities we computed earlier, we get the following:

$$
\begin{aligned}
&\mathbb{E}_{u\sim\mathbf{unif}(\mathbb{S}^{d-1})}\left[Var_{x\sim\mathbf{unif}(\mathcal{O}_u)}(\langle f,x\rangle)\right] \\
&= \mathbb{E}_{u\sim\mathbf{unif}(\mathbb{S}^{d-1})}\left[Var_{v\sim\mathbf{unif}(\mathcal{O}_f)}(\langle u,v\rangle)\right] \quad \text{applying equation 35} \\
&= \mathbb{E}_{u\sim\mathbf{unif}(\mathbb{S}^{d-1})}\left[u^T\mathbb{\Sigma}_f u\right] \quad \text{applying equation 38} \\
&= \mathbb{E}_{u\sim\mathbf{unif}(\mathbb{S}^{d-1})}\left[\sum_j \lambda_k(\alpha_j^{(u)})^2\right] \quad \text{applying equation 39} \\
&= \frac{\sum_j \lambda_j}{d} \quad \text{applying Lemma A.7} \\
&= \frac{\mathbb{T}\mathbb{r}(\mathbb{\Sigma}_f^{\mathbb{H}(T)})}{d} \\
&= \frac{\mathbb{V}_f^{\mathbb{H}(T)}}{d} \quad \text{reconciling with definition given by equation 11}
\end{aligned} \tag{40}
$$

□